

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
SUB CODE & NAME: CSA17- ARTIFICIAL INTELLIGENCE
LIST OF EXPERIMENTS

1. Write the python program to solve 8-Puzzle problem

Aim

To develop a Python program to solve the **8-Puzzle Problem** using the **A* Search Algorithm** by accepting the **initial state** and **goal state** from the user and finding the shortest sequence of moves.

Algorithm

1. Start the program.
2. Read the initial state from the user.
3. Read the goal state from the user.
4. Define the Manhattan Distance heuristic function.
5. Insert the initial state into the priority queue.
6. Repeat until the priority queue becomes empty:
 - Remove the state with the lowest heuristic cost.
 - If the current state is the goal state, display the solution path and stop.
 - Generate all valid neighboring states.
 - Calculate the heuristic value for each neighboring state.
 - Insert unvisited neighboring states into the priority queue.
7. If no solution is found, display "No Solution Exists."
8. Stop.

Program

```
import heapq
# Read puzzle state from user
def read_state(name):
    print(f"\nEnter the {name} (3 rows, space-separated):")
    state = []
    for i in range(3):
```

```

        row = list(map(int, input().split()))
        state.append(tuple(row))
    return tuple(state)

# Manhattan Distance Heuristic
def heuristic(state, goal):
    distance = 0
    goal_pos = {}
    # Store goal positions
    for i in range(3):
        for j in range(3):
            goal_pos[goal[i][j]] = (i, j)
    for i in range(3):
        for j in range(3):
            value = state[i][j]
            if value != 0:
                x, y = goal_pos[value]
                distance += abs(i - x) + abs(j - y)
    return distance

# Find blank tile (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate neighboring states
def get_neighbors(state):
    neighbors = []
    x, y = find_blank(state)
    moves = [(-1,0), (1,0), (0,-1), (0,1)]
    for dx, dy in moves:
        nx, ny = x + dx, y + dy
        if 0 <= nx < 3 and 0 <= ny < 3:
            new_state = [list(row) for row in state]

```



```

# ----- Main Program -----
start = read_state("Initial State")
goal = read_state("Goal State")
solution = a_star(start, goal)
if solution:
    print("\nSolution Found in", len(solution)-1, "moves.\n")
    for i, state in enumerate(solution):
        print("Step", i)
        print_state(state)
else:
    print("\nNo Solution Exists.")

```

Sample Input:

Enter the Initial State (3 rows, space-separated):

```

1 2 3
4 0 6
7 5 8

```

Enter the Goal State (3 rows, space-separated):

```

1 2 3
4 5 6
7 8 0

```

Sample output:

```
Solution Found in 2 moves.
```

```
Step 0
```

```
1 2 3
```

```
4 0 6
```

```
7 5 8
```

```
Step 1
```

```
1 2 3
```

```
4 5 6
```

```
7 0 8
```

```
Step 2
```

```
1 2 3
```

```
4 5 6
```

```
7 8 0
```

```
>>>
```

Result

The Python program was successfully implemented to solve the **8-Puzzle Problem** using the **A* Search Algorithm**. The program accepts the **initial state** and **goal state** from the user, computes the optimal solution using the Manhattan Distance heuristic, and displays the shortest sequence of moves along with all intermediate puzzle states until the goal state is reached.